Week 3

Learning NLP Basics

Guan-Yun Wang
Department of Information Management
Fu Jen Catholic University
2026.03.13

Dividing text into sentences

- When we work with text, we can work with text units on different scales: the document itself, such as a newspaper article, the paragraph, the sentence, or the word.
- Sentences are the main unit of processing in many NLP tasks.
    - For example, when we send data over to Large Language Models (LLMs), we frequently want to add some context to the prompt.
    - In some cases, we would like that context to include sentences from a text so that the model can extract some important information from that text.

Getting ready

The Adventures of Sherlock Holmes.

Download here

Prepare your materials

After you download the text, put it into a right folder, and you can read the text.

```
def read_text_file(filename):
    file = open(filename, "r", encoding = "utf-8")
    return file.read()
```

```
sherlock_holmes_part_of_text = read_text_file("../data/sherlock_holmes_1.txt")
print(sherlock_holmes_part_of_text)
```

Dividing sentences (by `nltk`)

- import the `nltk` package. If you run the code at the first time, you need to download tokenizer data.

```
import nltk
nltk.download('punkt') # Run this line only the first time
```

```
tokenizer = nltk.data.load("tokenizers/punkt/english.pickle")
sentences_nltk = tokenizer.tokenize(sherlock_holmes_part_of_text)
print(sentences_nltk)
print(len(sentences_nltk))
```

Observe the result, do you know how it divided a text into sentences? TronClass!

More information.

Dividing sentences (by `spaCy`)

```
import spacy
!python -m spacy download en_core_web_sm # Run this line only the first time
```

! means the code should be run in terminal.

```
nlp = spacy.load("en_core_web_sm") # here we're dealing with English
doc = nlp(sherlock_holmes_part_of_text)
sentences_spacy = [sentence.text for sentence in doc.sents]
print(sentences_spacy)
print(len(sentences_spacy))
```

Comparison between `nltk` and `spaCy`

TronClass!

Execute the codes and complete the sentence:

- An important difference between spaCy and NLTK is _____ it takes to complete the sentence-splitting process.
- The time that `nltk` / `spacy` uses is ____ times longer than `nltk` / `spacy`

the content:

```
print(sentences_nltk == sentences_spacy)
```

the time:

```
import time

def split_into_sentences_nltk(text):
    sentences = tokenizer.tokenize(text)
    return sentences

def split_into_sentences_spacy(text):
    doc = nlp(text)
    sentences = [sentence.text for sentence in doc.sents]
    return sentences

start = time.time()
split_into_sentences_nltk(sherlock_holmes_part_of_text)
print(f"NLTK: {time.time() - start} s")

start = time.time()
split_into_sentences_spacy(sherlock_holmes_part_of_text)
print(f"spaCy: {time.time() - start} s")
```

Comparison between `nltk` and `spaCy`

- An important difference between spaCy and NLTK is **the time** it takes to complete the sentence-splitting process.
- Reasons
  - spaCy loads a language model and uses several tools in addition to the tokenizer.
  - NLTK tokenizer has only one function: to separate the text into sentences.

Reflection:
Is it always good to use LLM/genAI to do everything for us?

- If you want to use only the tokenizer without other tools from spaCy. You can check their documentation for more information: https://spacy.io/usage/processing-pipelines

Other languages

for `nltk`

support Czech, Danish, Dutch, Estonian, Finnish, French, German, Greek, Italian, Norwegian,
Polish, Portuguese, Slovene, Spanish, Swedish, and Turkish

for example:

```
tokenizer = nltk.data.load("tokenizers/punkt/spanish.pickle")
```

for `spacy`

Here you can choose the language you want to use:
https://spacy.io/usage/models

Dividing sentences into words - Tokenization

- In many instances, we rely on individual words when we do NLP tasks.
- For example, when we build semantic models of texts by relying on the semantics of individual words, or when we are looking for words with a specific part of speech.
- We can also use NLTK and spaCy to do this task for us.

Prepare your materials and using `NLTK` to tokenize

- Read in the book snippet texts:

```
sherlock_holmes_part_of_text = read_text_file("../data/sherlock_holmes_1.txt")
print(sherlock_holmes_part_of_text)
```

- import the `nltk` package, and divide the input into words. The output of the function is a Python list of the words:

```
import nltk
words_nltk = nltk.tokenize.word_tokenize(sherlock_holmes_part_of_text)
print(words_nltk)
print(len(words_nltk))
```

Try: How NLTK divide the text like "*Hi, I'm Albert.*" ? What will the "*I'm*" be?

Custom tokenizer in NLTK package

- import the `MWETokenizer` class:

```
from nltk.tokenize import MWETokenizer
```

- Initialize the tokenizer and indicate that the words `dim sum dinner` should not be split:

```
tokenizer = MWETokenizer([('dim', 'sum', 'dinner')])
```

- Add more words that should be kept together:

```
tokenizer.add_mwe(('the', 'best', 'dim', 'sum'))
```

- Use the tokenizer to split a sentence:

```
tokens = tokenizer.tokenize('Last night I went for dinner in an Italian \
restaurant. The pasta was delicious.'.split())
print(tokens)
tokens = tokenizer.tokenize('I went out to a dim sum dinner last night. \
This restaurant has the best dim sum in town.'.split())
print(tokens)
```

Another way: Using `spacy` to tokenize

- Import the spacy package:

```
import spacy
```

- Initialize the spaCy engine using the English model:

```
nlp = spacy.load("en_core_web_sm")
```

- Divide the text into sentences and print the results

```
doc = nlp(sherlock_holmes_part_of_text)
words_spacy = [token.text for token in doc]
print(words_spacy)
print(len(words_spacy))
```

Compare the results from `nltk` and `spacy`

TronClass!

Execute the codes and complete the following sentences:

- You will notice that the length of the word list is **shorter/longer** when using spaCy than NLTK.
- One of the reasons is that spaCy keeps the newlines, and each newline is a separate token.
- The other difference is that spaCy splits words with a _____, such as *high-power*.

The content:

```
print(set(words_spacy)-set(words_nltk))
```

The time:

```
import time

def split_into_words_nltk(text):
    words = nltk.tokenize.word_tokenize(sherlock_holmes_part_of_text)
    return words

def split_into_words_spacy(text):
    doc = nlp(text)
    words = [token.text for token in doc]
    return words

start = time.time()
split_into_words_nltk(sherlock_holmes_part_of_text)
print(f"NLTK: {time.time() - start} s")

start = time.time()
split_into_words_spacy(sherlock_holmes_part_of_text)
print(f"spaCy: {time.time() - start} s")
```

Unfortunately, ...

- The NLTK package only has word tokenization for English.
- spaCy has models for other languages: Chinese, Dutch, English, French, German, Greek, Italian, Japanese, Portuguese, Romanian, Spanish, and others.

For more details, you can check their documentation:
https://spacy.io/usage/models

Part of speech tagging

- In many cases, NLP processing depends on determining the parts of speech of the words in the text.

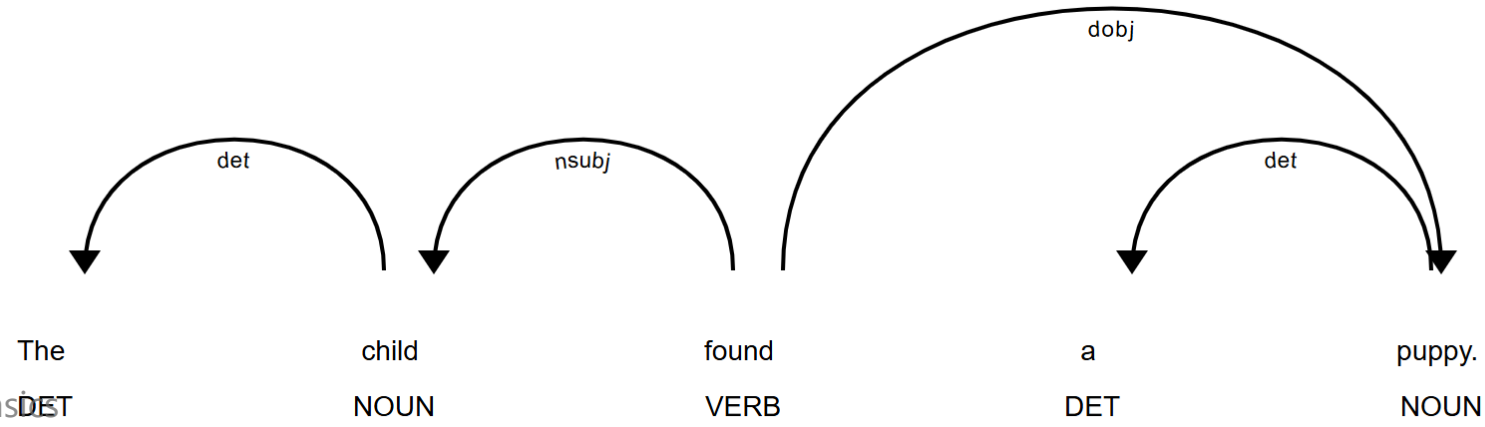
What is part of speech?

The list of part of speech tags: https://universaldependencies.org/u/pos/

ADJ: adjective
ADP: adposition
ADV: adverb
AUX: auxiliary
CCONJ: coordinating conjunction
DET: determiner
INTJ: interjection
NOUN: noun
NUM: numeral
PART: particle
PRON: pronoun
PROPN: proper noun
PUNCT: punctuation
SCONJ: subordinating conjunction
SYM: symbol
VERB: verb
X: other

What is part of speech?

- The structure of a sentence
- For example,
  - DET-N-V-DET-N (定詞-名詞-動詞-定詞-名詞)
  - The child found a puppy.
  - The professor wrote a book.
  - That runner won the race.



Define the function that will output parts of speech for every word. In this function, we first process the input text using the spaCy model that results in a `Document` object. The resulting `Document` object contains an iterator with `Token` objects, and each `Token` object has information about parts of speech.

```
def pos_tag_spacy(text, model):
    doc = model(text)
    words = [token.text for token in doc]
    pos = [token.pos_ for token in doc]
    return list(zip(words, pos))
```

Using `spacy` to analyze part-of-speech

Read your text:

```
text = read_text_file("[Your path here]/data/sherlock_holmes_1.txt")
```

Import spacy, use their small language model to run the function and print the output.

```
small_model = spacy.load("en_core_web_sm")
words_with_pos = pos_tag_spacy(text, small_model)
print(words_with_pos)
```

You can also run the visualization:

```
doc = small_model(text)
spacy.displacy.render(doc, style="dep")
```

Using `nltk` to analyze part-of-speech

Import nltk

```
import nltk
```

Download the part-of-speech model

```
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
```

Use a function so that we can compare later:

```
def pos_tag_nltk(text):
    words = nltk.tokenize.word_tokenize(text)
    words_with_pos = nltk.pos_tag(words, lang='eng')
    return words_with_pos
```

read your text

```
text = read_text_file("../data/sherlock_holmes_1.txt")
```

try your part of speech:

```
words_with_pos = pos_tag_nltk(text)
print(words_with_pos)
```

Note: the list of part of speech tags that NLTK uses is different from what SpaCy uses, and can be accessed by running the following commands:

```
python
>>> import nltk
>>> nltk.download('tagsets')
>>> nltk.help.upenn_tagset()
```

Compare running times for NLTK and spaCy

```
import time
start = time.time()
pos_tag_nltk(text)
print(f"NLTK: {time.time() - start} s")

start = time.time()
pos_tag_spacy(text, small_model)
print(f"spaCy: {time.time() - start} s")
```

Using LLM to help you analyze part-of-speech

- 他知道這件事不要緊
- 這份報告我寫不好
- 他有一個女兒，在醫院工作
- 我在樓上看到了他
- 兒的生活好痛苦也沒有糧食多病少掙了很多錢

TronClass!

Combining similar words - lemmatization 詞形還原

- We can find the canonical form of the word using lemmatization.
- For example, the lemma of the word *cats* is *cat*, and the lemma of the word *ran* is *run*

Let's deal with this issue with `spacy`

- Firstly, Create a list of words we want to lemmatize:

```
words = ["leaf", "leaves", "booking", "writing", "completed", "stemming"]
```

- Create a `document` object for each of the words.

```
docs = [small_model(word) for word in words]
```

- Print the words and their lemmas for each of the words in the list.

```
for doc in docs:
    for token in doc:
        print(token, token.lemma_)
```

Results:

```
leaf leaf
leaves leave
booking book
writing write
completed
complete
stemming stem
```

Besides, we can also give the spaCy continuous text instead of individual words, it is likely to **correctly disambiguate the words**.

```
text = read_text_file("../data/sherlock_holmes_1.txt")
doc = small_model(text)
for token in doc:
    print(token, token.lemma_)
```

Results:

```
To to
Sherlock Sherlock
Holmes Holmes
she she
is be
always always
_ _
the the
_ _
woman woman
. .
I I
have have
seldom seldom
heard hear
him he
```

Removing stopwords

- When we work with words, especially if we are considering the words' semantics, we sometimes need to exclude some very frequent words that do not bring any substantial meaning into the sentence
    - such as *but, can, we, etc*.
- For example, if we want to get a rough sense of the topic of a text, we could count its most frequent words. HOwever, the most frequent words will be stopwords.
- NLTK supports for stopwords: Arabic, Azerbaijani, Danish, Dutch, English, Finnish, French, German, Greek, Hungarian, Italian, Kazakh, Nepali, Norwegian, Portuguese, Romanian, Russian, Spanish, Swedish, and Turkish. (Unfortunately, no Chinese)

Remove stopwords using `spaCy`

Import spacy and load the model

```
import spacy
small_model = spacy.load("en_core_web_sm")
```

Tokenize the texts

```
def word_tokenize_spacy(text):
    doc = small_model(text)
    words = [token.text for token in doc]
    return words
```

Assign the stopwords to a variable for convenience:

```
stopwords = small_model.Defaults.stop_words
print(stopwords)
print(type(stopwords))
```

Here shows the original length of the texts

```
words = word_tokenize_spacy(text)
print(len(words))
```

Here shows the results that remove the stopwords

```
words = [word for word in words if word.lower() not in stopwords]
print(len(words))
```